

هياكل البيانات في البايثون

ما هو الـ Tuple ؟

الـ Tuple هو نوع من الحاويات (Containers) في بايثون يستخدم لتخزين مجموعة من العناصر مرتبة حسب ترتيب محدد. تخيله كصندوق صغير يمكنك وضع عدة أشياء بداخله، ولكن بمجرد إغلاقه لا يمكنك تغيير محتوياته!

الخصائص الرئيسية للـ Tuple:

- **مرتّب (Ordered):** العناصر تحتفظ بترتيب إضافتها
- **يسمح بالتكرار (Allows duplicates):** يمكن أن يحتوي على عناصر مكررة
- **غير قابل للتغيير (Immutable):** لا يمكن تعديله بعد الإنشاء

كيفية إنشاء Tuple

لإنشاء tuple ، نضع العناصر بين قوسين عاديين () ويفصل بينها بفواصل:

فارغ tuple إنشاء

```
empty_tuple = ()  
print(empty_tuple) # ()
```

يحتوي على أرقام tuple إنشاء

```
numbers = (1, 2, 3, 4, 5)  
print(numbers) # (1, 2, 3, 4, 5)
```

يحتوي على نصوص tuple إنشاء

```
fruits = ("برتقال", "موز", "تفاح")  
print(fruits) # ('برتقال', 'موز', 'تفاح')
```

مختلط tuple إنشاء

```
mixed = (1, "نص", 3.14, True)  
print(mixed) # (1, 'نص', 3.14, True)
```

ملاحظة مهمة: إذا كنت تريد إنشاء tuple يحتوي على عنصر واحد فقط، يجب إضافة فاصلة بعد العنصر:

tuple هذا سيكون مجرد رقم وليس

```
not_tuple = (5)  
print(type(not_tuple)) # <class 'int'>
```

صحيح بعنصر واحد tuple هذا هو

```
correct_tuple = (5,)   
print(type(correct_tuple)) # <class 'tuple'>
```

الوصول إلى عناصر الـ Tuple

يمكنك الوصول إلى عناصر الـ Tuple باستخدام الفهرس (Index) تماماً مثل القوائم:

tuple إنشاء

```
colors = ("أصفر", "أزرق", "أخضر", "أحمر")  
# الوصول إلى العنصر الأول (الفهرس 0)  
print(colors[0]) # أصفر
```

```
# الوصول إلى العنصر الثالث (الفهرس 2)
print(colors[2]) # أزرق
# الوصول إلى العنصر الأخير
print(colors[-1]) # أصفر
# tuple لاستخراج جزء من الـ slicing
print(colors[1:3]) # ('أخضر', 'أزرق')
```

ما الفرق بين List و Tuple ؟
هذا سؤال مهم جداً! إليك الفروق الرئيسية:

الميزة	Tuple	List
القابلية للتغيير	غير قابل للتغيير (Immutable)	قابل للتغيير (Mutable)
الاقواس المستخدمة	()	[]
الأداء	أسرع في التكرار	أبطأ قليلاً
الاستخدام	للبيانات الثابتة	للبيانات المتغيرة

مثال يوضح الفرق #

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
```

```
# list يمكنني تعديل الـ
my_list[0] = 10 # يعمل
print(my_list) # [10, 2, 3]
```

```
# tuple لا يمكنني تعديل الـ
# my_tuple[0] = 10 # خطأ! لا يعمل
```

العمليات الأساسية على Tuples

1 التكرار (Looping)

```
days = ("الاثنين", "الأحد", "السبت")
# for loop التكرار باستخدام
for day in days:
    print(day)
```

2 التحقق من وجود عنصر

```
fruits = ("برتقال", "موز", "تفاح")
# التحقق إذا كان العنصر موجوداً
print("تفاح" in fruits) # True
print("فراولة" in fruits) # False
```

3 دمج Tuples

```
tuple1 = (1, 2, 3)
tuple2 = ("a", "b", "c")
```

```
# دمج الـ tuples
combined = tuple1 + tuple2
print(combined) # (1, 2, 3, 'a', 'b', 'c')
```

4 تكرير الـ Tuple

```
numbers = (1, 2)
repeated = numbers * 3
print(repeated) # (1, 2, 1, 2, 1, 2)
```

الطول والعدد

```
# معرفة عدد العناصر
students = ("زينب", "محمد", "فاطمة", "أحمد")
print(len(students)) # 4
```

```
# عدد تكرار عنصر معين
numbers = (1, 2, 3, 2, 1, 2)
print(numbers.count(2)) # 3 (الرقم 2 يتكرر 3 مرات)
```

```
# معرفة موقع عنصر
print(numbers.index(3)) # 2 (الرقم 3 موجود في الفهرس 2)
```

#مثال: إرجاع أكثر من قيمة من دالة

```
def get_student_info():
    name = "أحمد"
    age = 20
    grade = "A"
    return name, age, grade # tuple هذا سيعود كـ
```

```
student_info = get_student_info()
print(student_info) # ('أحمد', 20, 'A')
```

نصائح عملية

1. استخدم الـ Tuples عندما تعلم أن البيانات لن تتغير
2. الـ Tuples أكثر أماناً من الـ Lists للبيانات الثابتة
3. يمكنك تحويل الـ List إلى Tuple والعكس
4. الـ Tuples أسرع في المعالجة من الـ Lists

tuple و list تحويل بين

```
my_list = [1, 2, 3]
my_tuple = tuple(my_list) # list إلى tuple تحويل
new_list = list(my_tuple) # tuple إلى list تحويل
```

ما هي المجموعات (Sets) ؟

المجموعة (Set) في بايثون هي مجموعة غير مرتبة من العناصر الفريدة (لا تسمح بالتكرار). تخزن البيانات بدون أي ترتيب محدد، مما يجعلها سريعة جداً في عمليات البحث والتحقق من وجود عنصر. خصائص المجموعات الأساسية:

- عناصر فريدة: لا يمكن أن تحتوي على عنصرين متماثلين.
- غير مرتبة: لا يمكنك الوصول للعناصر باستخدام الفهرس (Index) مثل القوائم.
- قابلة للتغيير (Mutable): يمكنك إضافة وحذف العناصر منها.
- تدعم أنواع بيانات مختلفة: يمكن أن تحتوي على أرقام، نصوص، وغيرها.

كيفية إنشاء مجموعة (Set) في بايثون

هناك طريقتان رئيسيتان لإنشاء مجموعة:

الطريقة الأولى: باستخدام الأقواس المعقوفة {}

إنشاء مجموعة من الأرقام

```
my_numbers = {1, 5, 3, 9, 7}
```

```
print(my_numbers) # {1, 3, 5, 7, 9} (تذكر: غير مرتبة!)
```

إنشاء مجموعة من النصوص

```
fruits = {"برتقال", "موز", "تفاح"}
```

```
print(fruits)
```

محاولة إنشاء مجموعة بعناصر مكررة

```
unique_set = {1, 2, 2, 3, 3, 3}
```

```
print(unique_set) # {1, 2, 3} سيطبع: {1, 2, 3} فقط! التكرارات تم حذفها تلقائياً
```

set() الطريقة الثانية: باستخدام الدالة

تحويل قائمة إلى مجموعة (لإزالة التكرارات)

```
my_list = [10, 20, 30, 20, 10]
```

```
converted_set = set(my_list)
```

```
print(converted_set) # {10, 20, 30}
```

{} وليس set() هـام: إنشاء مجموعة فارغة

```
empty_set = set()
```

```
print(type(empty_set)) # <class 'set'>
```

ملاحظة مهمة: {} تنشئ قاموساً فارغاً (Dictionary) ، وليس مجموعة فارغة. لإنشاء مجموعة فارغة، يجب استخدام set().

إضافة عناصر إلى المجموعة

يمكننا إضافة عناصر جديدة للمجموعة بطريقتين:

1 استخدام الدالة add()

```
colors = {"أزرق", "أحمر"}
colors.add("أخضر")
print(colors) # {'أزرق', 'أخضر', 'أحمر'}
```

محاولة إضافة عنصر موجود مسبقاً

```
colors.add("أحمر") # لأن العنصر موجود بالفعل
print(colors) # {'أزرق', 'أخضر', 'أحمر'}
```

2 استخدام الدالة update()

تستخدم update() لإضافة عدة عناصر مرة واحدة (من قائمة أو مجموعة أخرى).

```
my_set = {1, 2, 3}
```

إضافة عناصر من قائمة

```
my_set.update([4, 5, 6])
print(my_set) # {1, 2, 3, 4, 5, 6}
```

إضافة عناصر من مجموعة أخرى

```
another_set = {7, 8}
my_set.update(another_set)
print(my_set) # {1, 2, 3, 4, 5, 6, 7, 8}
```

حذف عناصر من المجموعة

هناك عدة طرق لحذف عنصر من مجموعة:

1 الدالة remove()

تحذف عنصراً معيناً، ولكن تسبب خطأ (Error) إذا كان العنصر غير موجود.

```
numbers = {100, 200, 300}
numbers.remove(200)
print(numbers) # {100, 300}
```

KeyError إذا فككت التعليق، سيتسبب هذا بسطر في خطأ `numbers.remove(999)`

2 الدالة discard()

تحذف عنصراً معيناً، ولكن لا تسبب أي خطأ إذا كان العنصر غير موجود.

```
numbers = {100, 200, 300}
numbers.discard(200)
print(numbers) # {100, 300}
```

لن يسبب أي خطأ حتى لو العنصر غير موجود `numbers.discard(999)`

```
print(numbers) # {100, 300} (لم يتغير)
```

3 الدالة pop()

تُحذف وتُعيد عنصراً عشوائياً من المجموعة (تذكر: المجموعات غير مرتبة).

```
letters = {"A", "B", "C", "D"}
removed_item = letters.pop()
print(f"العنصر المحذوف: {removed_item}")
print(f"المجموعة بعد الحذف: {letters}")
```

4 الدالة clear()

تُفرغ المجموعة بالكامل من جميع عناصرها.

```
my_set = {1, 2, 3, 4, 5}
my_set.clear()
print(my_set) # set() (مجموعة فارغة)
```

العمليات الأساسية على المجموعات

هذه العمليات هي ما يجعل المجموعات مفيدة جداً! دعنا نتخيل أن لدينا مجموعتين من الطلاب.

```
# مجموعة طلاب فصل الرياضيات
math_students = {"أحمد", "سارة", "خالد", "فاطمة"}
# مجموعة طلاب فصل الفيزياء
physics_students = {"علي", "نورة", "خالد", "سارة"}
```

1 الاتحاد | (Union) أو union()

النتيجة: جميع الطلاب الذين يدرسون في أي من الفصلين (بدون تكرار).

```
all_students = math_students | physics_students
# أو: all_students = math_students.union(physics_students)
print(all_students) # {'أحمد', 'سارة', 'خالد', 'فاطمة', 'نورة', 'علي'}
```

2 التقاطع & (Intersection) أو intersection()

النتيجة: فقط الطلاب الذين يدرسون في كلا الفصلين.

```
common_students = math_students & physics_students
# أو: common_students = math_students.intersection(physics_students)
print(common_students) # {'سارة', 'خالد'}
```

3 الفرق - (Difference) أو difference()

النتيجة: الطلاب الموجودين في المجموعة الأولى وليسوا في المجموعة الثانية.

```
only_math = math_students - physics_students
# أو: only_math = math_students.difference(physics_students)
print(only_math) # {'أحمد', 'فاطمة'} (يدرسون رياضيات فقط)
```

```
only_physics = physics_students - math_students
print(only_physics) # {'نورة', 'علي'} (يدرسون فيزياء فقط)
```

اختبر فهمك: مثال عملي بسيط
لنحل مشكلة شائعة باستخدام المجموعات: إزالة التكرارات من قائمة.

لدينا قائمة تحتوي على أرقام مكررة

```
duplicate_list = [5, 2, 8, 2, 5, 9, 8, 1]
```

الخطوة 1: تحويل القائمة إلى مجموعة (لإزالة التكرارات تلقائياً)

```
unique_set = set(duplicate_list)
```

```
print("المجموعة بعد إزالة التكرارات:", unique_set)
```

الخطوة 2: إذا أردنا النتيجة كقائمة مرة أخرى

```
final_list = list(unique_set)
```

```
print("القائمة النهائية بدون تكرارات:", final_list)
```

ملاحظة: ترتيب العناصر في القائمة النهائية قد يتغير لأن المجموعة غير مرتبة.

ما هو القاموس (Dictionary) ؟

القاموس هو هيكل بيانات يستخدم لتخزين مجموعة من القيم، ولكن بدلاً من استخدام الأرقام كفهارس (مثل القوائم)، يستخدم القاموس مفاتيح (Keys). يمكنك التفكير فيه كقاموس لغوي حقيقي: تبحث عن كلمة (المفتاح) لتحصل على معناها (القيمة).

الخصائص الرئيسية للقاموس:

- غير مرتب (في إصدارات بايثون القديمة: (3.7 < لا يتم تخزين العناصر بترتيب محدد.
- قابل للتغيير (Mutable): يمكنك إضافة عناصر جديدة، إزالتها، أو تعديلها بعد الإنشاء.
- لا يسمح بتكرار المفاتيح: كل مفتاح في القاموس يجب أن يكون فريداً. إذا أضفت مفتاحاً موجوداً مسبقاً، فسيتم تحديث قيمته.

كيفية إنشاء قاموس

أسهل طريقة لإنشاء قاموس هي وضع عناصره داخل أقواس معقوفة {}, حيث يكون كل عنصر على صورة **مفتاح: قيمة**، ويفصل بين العناصر بفاصلة.

إنشاء قاموس فارغ

```
my_empty_dict = {}
```

إنشاء قاموس لتخزين معلومات طالب

```
student = {  
    "name": "أحمد",  
    "age": 20,  
    "major": "هندسة البرمجيات",  
    "GPA": 3.75  
}
```

طباعة القاموس كاملاً

```
print(student)
```

ماذا يحدث في الكود أعلاه؟ قمنا بإنشاء قاموس student يحتوي على 4 عناصر. المفتاح "name" قيمته "أحمد"، والمفتاح "age" قيمته 20، وهكذا.

الوصول إلى القيم داخل القاموس

لا يمكنك استخدام الأرقام كفهرس للوصول للبيانات كما في القوائم. بدلاً من ذلك، استخدم المفتاح الموضوع داخل أقواس مربعة [].

```
student = {"name": "أحمد", "age": 20, "major": "هندسة البرمجيات"}
```

الوصول إلى قيمة المفتاح "name"

```
print(student["name"]) # الناتج: أحمد
```

الوصول إلى قيمة المفتاح "age"

```
print(student["age"]) # الناتج: 20
```

ما الخطأ في هذا الكود؟

```
print(student["phone_number"]) # سيتسبب في خطأ!
```

سيظهر خطأ `KeyError` لأن المفتاح `"phone_number"` غير موجود في القاموس. لحل هذه المشكلة، يمكننا استخدام الطريقة `.get()`.

الطريقة الآمنة: استخدام `.get()`.

تتيح لك هذه الطريقة الوصول إلى قيمة مفتاح معين، وإرجاع قيمة افتراضية (مثل `None`) إذا لم يكن المفتاح موجوداً، مما يمنع حدوث الأخطاء.

البحث عن مفتاح موجود

```
print(student.get("name")) # الناتج: أحمد
```

البحث عن مفتاح غير موجود

```
print(student.get("phone_number")) # الناتج: None (لا شيء، ولكن بدون خطأ)
```

يمكنك تحديد قيمة افتراضية بديلة

```
print(student.get("phone_number", "لم يتم إضافة رقم")) # الناتج: لم يتم إضافة رقم
```

إضافة عناصر جديدة أو تعديل القيم

لإضافة عنصر جديد، كل ما عليك فعله هو تعيين قيمة لمفتاح جديد.

```
student = {"name": "أحمد", "age": 20}
```

(غير موجود "major" المفتاح) إضافة عنصر جديد

```
student["major"] = "هندسة البرمجيات"
```

```
print(student) # الناتج: {'name': 'أحمد', 'age': 20, 'major': 'هندسة البرمجيات'}
```

(موجود "age" المفتاح) تعديل قيمة عنصر موجود

```
student["age"] = 21
```

```
print(student) # الناتج: {'name': 'أحمد', 'age': 21, 'major': 'هندسة البرمجيات'}
```

حذف العناصر من القاموس

يمكنك حذف عنصر باستخدام الكلمة المفتاحية `del`.

```
student = {"name": "أحمد", "age": 20, "major": "هندسة البرمجيات"}
```

"age" حذف العنصر الذي مفتاحه

```
del student["age"]
```

```
print(student) # الناتج: {'name': 'أحمد', 'major': 'هندسة البرمجيات'}
```

أهم الطرق (Methods) الأساسية للقواميس

1. `keys()` تُرجع قائمة تحتوي على جميع المفاتيح في القاموس.
2. `values()` تُرجع قائمة تحتوي على جميع القيم في القاموس.
3. `items()` تُرجع قائمة تحتوي على جميع العناصر على شكل `tuples` (مفتاح، قيمة)

```
student = {"name": "أحمد", "age": 20, "major": "هندسة البرمجيات"}
```

```
# الحصول على جميع المفاتيح
```

```
print(student.keys()) # الناتج: dict_keys(['name', 'age', 'major'])
```

```
# الحصول على جميع القيم
```

```
print(student.values()) # الناتج: dict_values(['أحمد', 20, 'هندسة البرمجيات'])
```

```
# الحصول على جميع العناصر (مفيد جداً للتكرار)
```

```
print(student.items()) # الناتج: dict_items([('name', 'أحمد'), ('age', 20), ('major', 'هندسة  
البرمجيات')])
```

مثال تطبيقي بسيط: دفتر عناوين

لنطبق ما تعلمناه في مثال واقعي. سننشئ دفتر عناوين بسيطاً.

```
# إنشاء دفتر عناوين
```

```
address_book = {  
    "أميرة": "0551234567",  
    "خالد": "0507654321"  
}
```

```
# إضافة صديق جديد
```

```
address_book["سارة"] = "0540001111"
```

```
# البحث عن رقم خالد
```

```
print("رقم خالد هو:", address_book.get("خالد"))
```

```
# حذف جهة اتصال
```

```
del address_book["أميرة"]
```

```
# طباعة جميع الأسماء (المفاتيح) في الدفتر
```

```
print("جهات الاتصال لدي")
```

```
for name in address_book.keys():
```

```
    print("-", name)
```

ما هو الـ Stack (المكدس)؟

الستاك هو هيكل بيانات خطي (Linear Data Structure) يعمل بمبدأ LIFO، وهي اختصار لـ "Last In, First Out"، أي "آخر من يدخل هو أول من يخرج".
تخيله هكذا:

- مكدس الكتب: تضيف كتاباً جديداً في الأعلى (يدخل أخيراً)، وعندما تريد كتاباً، تأخذه من الأعلى (يخرج أولاً).
- مكدس الأطباق: كما ذكرنا سابقاً.
- زر الرجوع (Undo) في برامج الكتابة: آخر إجراء قمت به هو أول إجراء يمكنك التراجع عنه.

العمليات الأساسية على الـ Stack

لعمل الستاك بشكل صحيح، نحتاج إلى ثلاث عمليات رئيسية:

1. **push(item)** إضافة عنصر: تضيف عنصراً جديداً إلى قمة (أعلى) المكدس.
2. **pop()** إزالة عنصر: تزيل العنصر الموجود في قمة المكدس وتعيده.
3. **peek()** نظرة خاطفة: تُرجع العنصر الموجود في قمة المكدس دون إزالته، فقط لمعرفة ما هو.

طريقتان لتنفيذ الـ Stack في بايثون

هناك طريقتان رئيسيتان لاستخدام الستاك في بايثون: الطريقة البسيطة باستخدام القوائم الجاهزة، والطريقة الاحترافية بالتنفيذ من الصفر.

الطريقة 1: استخدام القوائم المدمجة (الطريقة البسيطة)

بايثون لا تحتوي على هيكل Stack مدمج، ولكن يمكننا بسهولة محاكاته باستخدام القوائم (Lists) هيا نطبق.

المثال 1: إنشاء ستاك وإضافة عناصر (push)

إنشاء ستاك فارغ باستخدام قائمة #

هذا هو ستاكنا # my_stack = []

إضافة عناصر إلى قمة الستاك: push عملية #

العنصر الأول يصبح في القاع # ("كتاب الرياضيات") my_stack.append("كتاب الرياضيات")

my_stack.append("كتاب العلوم")

هذا العنصر الأخير سيكون في القمة # ("كتاب التاريخ") my_stack.append("كتاب التاريخ")

print("push: الستاك بعد عمليات", my_stack)

المخرجات: ["كتاب الرياضيات", "كتاب العلوم", "كتاب التاريخ"] #

الشرح: استخدمنا الدالة append() لإضافة إلى نهاية القائمة، والتي تمثل قمة الستاك لدينا.

المثال 2: إزالة عنصر من الستاك (pop)

المتابعة من المثال السابق #

print("الستاك الحالي:", my_stack) # ["كتاب الرياضيات", "كتاب العلوم", "كتاب التاريخ"]

إزالة العنصر من القمة: pop عملية #

top_item = my_stack.pop()

print("العنصر الذي تمت إزالته (pop):", top_item) # كتاب التاريخ

print("الستاك بعد عملية pop:", my_stack) # ["كتاب الرياضيات", "كتاب العلوم"]

```

أخرى pop لنقم بعملية #
removed_item = my_stack.pop()
print("تمت إزالة:", removed_item) # كتاب العلوم
print("الستاك الآن:", my_stack) # [كتاب الرياضيات]
الشرح: الدالة pop() تزيل آخر عنصر في القائمة (قمة الستاك) وتعيد قيمته. لاحظ كيف يخرج "كتاب التاريخ" أولاً
لأنه دخل أخيراً مبدأً (LIFO).
المثال 3: النظر إلى العنصر العلوي دون إزالته (peek)

```

```

# المتابعة من المثال السابق
print("الستاك الحالي:", my_stack) # [كتاب الرياضيات]

```

```

# رؤية العنصر في القمة دون حذفه: peek عملية #
نتحقق أولاً أن الستاك ليس فارغاً #
if my_stack:
    top_element = my_stack[-1] # (القمة) عنصر
    print("الستاك الرياضي:", top_element) # (peek) العنصر في قمة الستاك
    print("الستاك الرياضي (لم يتغير) peek:", my_stack) # [كتاب الرياضيات]
else:
    print("الستاك فارغ")

```

الشرح: للقيام بعملية peek، نصل إلى العنصر باستخدام الفهرس -1، والذي يشير دائماً إلى آخر عنصر في القائمة (أعلى الستاك)، دون استخدام pop() حتى لا نحذفه.

الطريقة 2: تنفيذ الـ Stack من الصفر (الطريقة الاحترافية)
 للفهم الأعمق لكيفية عمل الستاك داخلياً، دعنا ننشئ فئة Stack خاصة بنا من البداية.

```

class Stack:
    def __init__(self):
        """منشئ الفئة - ينشئ ستاك فارغ"""
        self.items = [] # نستخدم قائمة لتخزين العناصر

    def push(self, item):
        """إضافة عنصر إلى قمة الستاك"""
        self.items.append(item) # نضيف العنصر إلى نهاية القائمة

    def pop(self):
        """إزالة وإرجاع العنصر من قمة الستاك"""
        if self.is_empty():
            raise Exception("الستاك فارغ لا يمكن إجراء")
        return self.items.pop() # نزيل آخر عنصر من القائمة

    def peek(self):
        """إرجاع العنصر في قمة الستاك دون إزالته"""

```

```

if self.is_empty():
    raise Exception("، الستاك فارغاً # !، الستاك فارغاً لا يمكن إجراء")
return self.items[-1] # نعود بقيمة آخر عنصر دون حذفه

```

```

def is_empty(self):
    """التحقق مما إذا كان الستاك فارغاً"""
    return len(self.items) == 0 # إذا كان الطول يساوي صفر True نعود بـ

```

```

def size(self):
    """إرجاع عدد العناصر في الستاك"""
    return len(self.items) # نعود بطول القائمة

```

مثال على استخدام الفئة المخصصة

```

print("=== المخصص Stack استخدام ال ===")
book_stack = Stack()

```

إضافة عناصر إلى الستاك

```

book_stack.push("رواية 1984")
book_stack.push("قصة الحيوانات")
book_stack.push("كتاب البرمجة")

```

```

print("حجم الستاك:", book_stack.size()) # 3
print("كتاب البرمجة #", book_stack.peak()) "العنصر في القمة"

```

إزالة عنصر

```

removed_book = book_stack.pop()
print("تمت إزالة:", removed_book) # كتاب البرمجة
print("قصة الحيوانات #", book_stack.peak()) "في القمة الآن العنصر"
print("pop: حجم الستاك بعد", book_stack.size()) # 2

```

الشرح: هنا قمنا ببناء هيكل الستاك بشكل كامل من الصفر. الفائدة من هذه الطريقة هي الفهم العمق لكيفية عمل الستاك وإمكانية تخصيصه حسب احتياجاتك.

تطبيقات عملية إضافية للـ Stack

لنرى كيف يمكن استخدام الستاك في مواقف حقيقية:

في برنامج نصي (Undo) مثال 1: محاكاة زر الرجوع

```

class TextEditor:
    def __init__(self):
        self.text = ""
        self.undo_stack = Stack() # ستاك لحفظ الحالات السابقة

```

```

def write(self, new_text):
    """إضافة نص جديد وحفظ الحالة السابقة"""
    self.undo_stack.push(self.text) # حفظ الحالة الحالية قبل التعديل
    self.text += new_text

def undo(self):
    """التراجع عن آخر عملية"""
    if not self.undo_stack.is_empty():
        self.text = self.undo_stack.pop() # استعادة الحالة السابقة

# اختبار محرر النص
editor = TextEditor()
editor.write("مرحبا")
editor.write("بالعالم")
print("النص الحالي:", editor.text) # مرحبا بالعالم

editor.undo()
print("بعد التراجع:", editor.text) # مرحبا

# مثال 2: التحقق من توازن الأقواس
def check_parentheses(expression):
    """التحقق مما إذا كانت الأقواس متوازنة في التعبير"""
    stack = Stack()
    opening_brackets = "({["
    closing_brackets = ")}]"
    matching = {'(': ')', '{': '}', '[': ']'}

    for char in expression:
        if char in opening_brackets:
            stack.push(char) # نضيف الأقواس الافتتاحية إلى الستاك
        elif char in closing_brackets:
            if stack.is_empty() or stack.pop() != matching[char]:
                return False # الأقواس غير متوازنة

    return stack.is_empty() # يجب أن يكون الستاك فارغاً إذا كانت الأقواس متوازنة

# اختبار توازن الأقواس
print("التوازن الصحيح:", check_parentheses("(a + b) * (c - d)")) # True
print("التوازن الخاطئ:", check_parentheses("((a + b) * (c - d)")) # False

```

التحقق من الستك الفارغ

من المهم جداً التحقق من أن الستك ليس فارغاً قبل إجراء عمليتي pop() أو peek.

المثال مع القوائم المدمجة

```
stack_example = []
```

من ستاك فارغ ستسبب خطأ pop محاولة

stack_example.pop() سيثير خطأ: IndexError: pop from empty list

الطريقة الصحيحة: التحقق أولاً

إذا كان الستك فارغاً: # if not stack_example:

print("⚠️، الستك فارغ pop تحذير: لا يمكن إجراء")

else:

item = stack_example.pop()

print("تمت إزالة", item)

ملخص

- الستك (Stack) هو هيكل بيانات يعمل بمبدأ LIFO (آخر من يدخل هو أول من يخرج)
- العمليات الأساسية هي push(): للإضافة، pop(): للإزالة، و peek(): للمعاينة.
- طريقتا التنفيذ: استخدام القوائم الجاهزة أو بناء فئة مخصصة من الصفر.
- التطبيقات العملية: أزرار الرجوع (Undo)، التحقق من توازن الأقواس، وغيرها.
- تأكد دائماً من أن الستك ليس فارغاً قبل استدعاء pop() أو peek().

ما هي قائمة الانتظار (Queue) ؟

قائمة الانتظار (Queue) هي هيكل بيانات خطي يتبع مبدأ أول ما يدخل هو أول ما يخرج (FIFO - First-In, First-Out). هذا يعني أن العنصر الذي تمت إضافته أولاً إلى القائمة سيكون هو أول عنصر يتم إزالته منها. العمليات الأساسية لقائمة الانتظار:

- **الإضافة (Enqueue):** إضافة عنصر جديد إلى نهاية قائمة الانتظار.
- **الإزالة (Dequeue):** إزالة العنصر الموجود في مقدمة قائمة الانتظار.
- **معاينة المقدمة (Peek/Front):** الحصول على قيمة العنصر في المقدمة دون إزالته.
- **الفحص (isEmpty):** التحقق مما إذا كانت قائمة الانتظار فارغة.

تنفيذ قائمة انتظار من الصفر (Custom Queue)

فهم كيفية بناء هيكل البيانات بنفسك هو أفضل طريقة لإتقان مبادئه. دعونا ننشئ صنف Queue خاص بنا.

1 هيكل الصنف الأساسي

سنبدأ بإنشاء الصنف وتهيئة القائمة الداخلية التي ستخزن البيانات.

```
class Queue:
    def __init__(self):
        # قائمة داخلية لتخزين عناصر قائمة الانتظار
        self.items = []

    def is_empty(self):
        """للتحقق مما إذا كانت قائمة الانتظار فارغة"""
        return len(self.items) == 0
```

تنفيذ العمليات الأساسية على قوائم الانتظار

الآن سنقوم بتطبيق العمليات الرئيسية: الإضافة (Enqueue)، والإزالة (Dequeue)، والمعاينة (Peek).

1 إضافة عناصر (Enqueue)

تتم الإضافة دائماً إلى نهاية قائمة الانتظار.

```
class Queue:
    # (الكود السابق) ...

    def enqueue(self, item):
        """إضافة عنصر جديد إلى نهاية قائمة الانتظار"""
        self.items.append(item) # نستخدم append إلى النهاية
        print(f"تمت إضافة {item}") # طباعة تأكيد للإضافة
```

2 إزالة عنصر (Dequeue)

تتم الإزالة دائماً من مقدمة قائمة الانتظار (العنصر الأقدم).

```
class Queue:
    # (الكود السابق) ...

    def dequeue(self):
        """إزالة العنصر من مقدمة قائمة الانتظار وإرجاعه"""
```

```

if self.is_empty():
    print("تحذير: قائمة الانتظار فارغة")
    return None # تعليق: إرجاع قيمة فارغة إذا كانت القائمة فارغة
return self.items.pop(0) # لإزالة العنصر الأول pop(0) نستخدم

```

3 معاينة العنصر الأمامي (Peek)

ننظر إلى العنصر الموجود في المقدمة دون إزالته من القائمة.

```

class Queue:
    # (الكود السابق) ...

    def peek(self):
        """إرجاع العنصر الموجود في مقدمة قائمة الانتظار دون إزالته"""
        if self.is_empty():
            print("قائمة الانتظار فارغة، لا يوجد عنصر للمعاينة")
            return None
        return self.items[0] # التعليق: الوصول للعنصر الأول مباشرة

    def size(self):
        """إرجاع عدد العناصر في قائمة الانتظار"""
        return len(self.items)

```

مثال تطبيقي: محاكاة طابور خدمة العملاء

لنرى كيف يعمل تنفيذنا الجديد في مثال حيوي.

```

# إنشاء كائن من قائمة الانتظار الخاصة بنا
customer_queue = Queue()

# إضافة عملاء إلى قائمة الانتظار
print("--- وصول العملاء ---")
customer_queue.enqueue("أحمد")
customer_queue.enqueue("سارة")
customer_queue.enqueue("خالد")

print(f"\nالعنصر التالي للخدمة: {customer_queue.peek()}")
print(f"عدد العملاء في الطابور: {customer_queue.size()}")

# خدمة العملاء
print("\n--- بدء الخدمة ---")
served = customer_queue.dequeue()
print(f"تم خدمة العميل: {served}")
print(f"بعد الخدمة، التالي هو {customer_queue.peek()}")

```

```

# إضافة عميل جديد أثناء الخدمة
print("\n--- وصول عميل جديد ---")
customer_queue.enqueue("نورة")
print(f"عدد العملاء الآن: {customer_queue.size()}")

# خدمة باقي العملاء
print("\n--- استكمال الخدمة ---")
while not customer_queue.is_empty():
    served = customer_queue.dequeue()
    print(f"تم خدمة: {served}")

print(f"\n؟الحالة النهائية. هل الطابور فارغ {customer_queue.is_empty()}")

```

استخدام القوائم المدمجة (List) مباشرة

للمقارنة، هذه هي الطريقة السريعة لاستخدام القائمة (list) كقائمة انتظار دون إنشاء صنف مخصص.

```

# إنشاء قائمة انتظار فارغة
my_queue = []

# إضافة عناصر (Enqueue)
my_queue.append("Task 1")
my_queue.append("Task 2")

# إزالة عنصر (Dequeue)
next_task = my_queue.pop(0) # إزالة العنصر الأول
print("المهمة التالية:", next_task)

# معاينة (Peek)
if my_queue: # التحقق من أن القائمة ليست فارغة
    print("العنصر في المقدمة:", my_queue[0])

```

استخدام وحدة queue الرسمية في بايثون

بينما القوائم (list) وتنفيذنا المخصص يعملان، إلا أن استخدام pop(0) غير فعال للبيانات الضخمة. توفر بايثون وحدة queue مع صنف Queue الأمثل.

```

import queue

# إنشاء كائن Queue
my_better_queue = queue.Queue()

# الإضافة (Enqueue)
my_better_queue.put("ملف 1")

```

```
my_better_queue.put("ملف 2")
```

```
# الإزالة (Dequeue)
```

```
processed_item = my_better_queue.get()
```

```
print("تم معالجة:", processed_item)
```

```
# للتحقق من الفراغ
```

```
print("هل فارغة؟", my_better_queue.empty())
```

ملاحظة queue.Queue: مصمم للبرمجة المتوازية. (thread-safe) للمشاريع التعليمية والبسيطة، التنفيذ المخصص أو استخدام القوائم مباشرة كافٍ.

ملخص سريع ومقارنة

- المبدأ FIFO: (أول ما يدخل هو أول ما يخرج)
- التنفيذ المخصص: بناء صنف Queue بأساليب enqueue()، dequeue() و peek() يعمق الفهم.
- باستخدام List مباشرة append(): للإضافة، pop(0) للإزالة (انتبه للأداء).
- الوحدة الرسمية queue.Queue: هو الحل الأمثل والأمن للتطبيقات الحقيقية والمعقدة.